

AUDITORIA DE APLICAÇÃO · 5 CATEGORIAS

Relatório de Auditoria de Segurança — FitFlow Home

Data: 29 de agosto de 2026 · **Escopo:** código em `/home/ruda/Projetos/fitflow_home` (`src/`, `supabase/migrations/`, `configs de deploy`) e `advisors/grants` do projeto Supabase remoto.

Nota metodológica. Stack detectada: TypeScript, Next.js 16 App Router, React 19, Tailwind 4, Supabase Auth + Postgres com RLS (sem ORM; cliente PostgREST via `supabase-js` / `@supabase/ssr`). Deploy: Vercel. Sem Docker, CI, Helm ou Terraform. Sem repositório git local — histórico de commits não pôde ser varrido. Mapeamento das categorias:

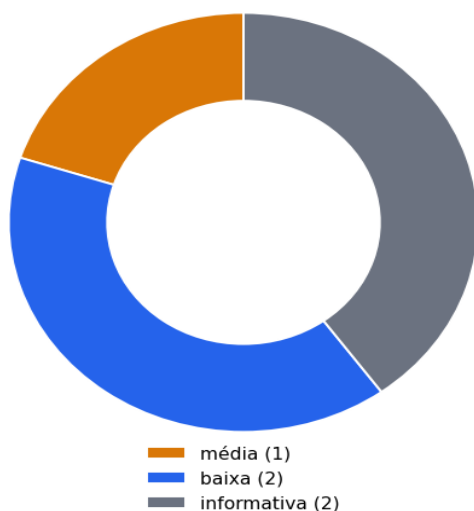
- 1 **1. Banco sem tranca** → RLS + policies + filtro `user_id/auth.uid()` nas server actions e em `data.ts`.
- 2 **2. Permissão no navegador** → busca por papéis (`isAdmin`, `canEdit`, `role`). Não há modelo de admin; todos autenticados têm o mesmo privilégio.
- 3 **3. IDOR** → único route handler de backend: `GET /auth/callback`; demais escritas são Server Actions; páginas (app) lidas uma a uma.
- 4 **4. Chaves expostas** → grep em `src/`, `migrations`, `README`, `DEPLOY`, `vercel.json`, `.env.example`. Bundle: só a anon key pública (esperado). Sem `.git`.
- 5 **5. XSS** → `dangerouslySetInnerHTML`, `href/src` dinâmicos, `eval`, `markdown`. Sem lib de sanitização no `package.json`.

Achados apenas quando verificados no código ou no banco ao vivo. Sem especulação de ataque sem sink.

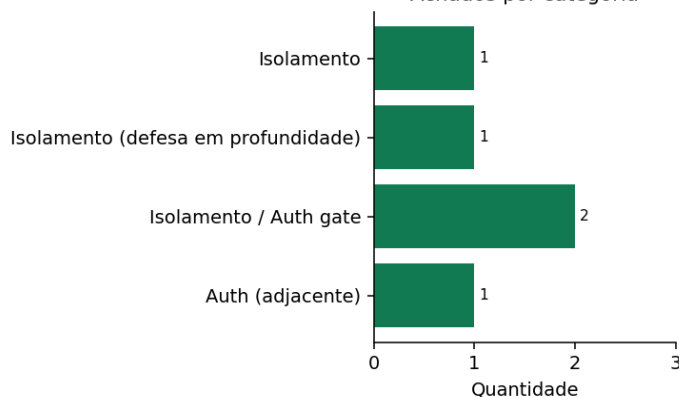
1. Resumo executivo

Foram registrados **5 achados**: crítica 0, alta 0, média 1, baixa 2, informativa 2. Nenhum crítico ou alto. O isolamento de dados do usuário (RLS + actions) está correto. O achado mais relevante é configuração de senha no Auth (F1).

Achados por severidade



Achados por categoria



2. Pontos fortes e pontos fracos

Pontos fortes (verificados)

- **RLS ligado** em `profiles`, `exercises`, `workouts`, `workout_exercises`, `workout_logs`, `body_metrics`. Políticas de dados do usuário usam `auth.uid()` (`select/update/insert/delete` próprios). Catálogo só `SELECT` para `authenticated` — sem policy de escrita = `deny`.
- **Triggers de dono** em `004_security_hardening.sql:39-81` forçam `user_id := auth.uid()` em `body_metrics` e `workout_logs` (spoof de `user_id` no insert é ignorado).
- **Server actions** (`src/lib/actions.ts`): as quatro funções chamam `requireAuthUser()` + `getUser()`. `updateProfile/reset` filtram `.eq('id'|'user_id', user.id)`. `saveWorkoutLog` valida UUID do treino de catálogo.
- **Páginas com dados** `/hoje`, `/historico`, `/perfil`, `/treino/[code]` chamam `requireUser()` e passam só `user.id` para `getProfile/getWorkoutLogs/getBodyMetrics`. RLS impede listar outro inquilino mesmo se o parâmetro mudasse.
- **IDOR**: nenhum handler busca objeto de usuário por ID de `path/query/body` sem posse. `/treino/[code]` só aceita A|B (catálogo). `/auth/callback` sanitiza `next` (obrigatório começar com `/` e não `//`) — `route.ts:4-8`.
- **Categoria 2 N/A**: não há `isAdmin/canEdit/role` no frontend. Resetar dados é UI de confirmação; o servidor revalida sessão em `resetUserData`.
- **Chaves**: sem `service_role`, `JWT` ou senha `hardcoded` em `src/`, `SQL`, `vercel.json`, `.env.example` (placeholders). `.gitignore` inclui `.env*`. Anon key em `NEXT_PUBLIC_` é o modelo do Supabase.
- **XSS**: único `dangerouslySetInnerHTML` é script estático de tema (`layout.tsx:54-67`, chave fixa `fitflow-theme`). `display_name` e erros do Auth entram como texto React (escapados). Sem `eval/markdown/v-html`. Sem e-mails HTML no repo.
- **handle_new_user**: `EXECUTE` só `postgres/service_role` — não chamável por `anon/authenticated`.

- **Advisor RLS:** get_advisors(security) não apontou tabela sem RLS; só senha vazada.

Pontos fracos (riscos centrais)

- Auth aceita senhas já vazadas (F1) — único achado de média.
- Cinto de autenticação do Next depende de env + requireUser por página; o proxy falha aberto e o layout (app) não fecha o segundo cinto (F2, F3).
- Grants padrão do Supabase para anon são amplos; o isolamento é 100% RLS (F4).
- Sem git local: não foi possível varrer histórico de segredos commitados.
- Sem Docker/CI/Helm/Terraform nesta pasta — categoria 4 nesses artefatos não se aplica.

3. Achados detalhados por categoria

Cada linha foi conferida no arquivo (ou no banco/advisor). Paleta: crítica #B91C1C · alta #EA580C · média #D97706 · baixa #2563EB · informativa #6B7280.

Sev.	Arquivo:linha	Descrição
MÉDIA	Supabase Auth (projeto remoto):—	F1 · Proteção contra senhas vazadas desativada O advisor de segurança do Supabase confirma que Leaked Password Protection está off. Usuários podem cadastrar senhas já comprometidas (HaveIBeenPwned). <i>Explorável:</i> Qualquer pessoa no signup ou troca de senha pode usar uma senha vazada conhecida, desde que tenha ≥ 8 caracteres (único check no app). <i>Condição:</i> Configuração atual do projeto Auth.
BAIXA	src/lib/supabase/proxy.ts:10–12	F2 · Proxy de sessão falha aberto sem env Se NEXT_PUBLIC_SUPABASE_URL ou NEXT_PUBLIC_SUPABASE_ANON_KEY estiverem ausentes, o proxy devolve NextResponse.next() sem checar login. <i>Explorável:</i> Rotas do grupo (app) passam no proxy. Hoje/histórico/perfil ainda chamam requireUser(); /descanso não. Em produção atual as env existem. <i>Condição:</i> Deploy ou runtime sem as variáveis públicas do Supabase.
BAIXA	src/app/(app)/layout.tsx e src/app/(app)/descanso/page.tsx:1–18 e 8–47	F3 · Layout autenticado sem requireUser; /descanso sem sessão O layout (app) não valida sessão. A página /descanso não chama requireUser. A proteção depende só do proxy. <i>Explorável:</i> Com o fail-open do F2, anônimo acessa /descanso (conteúdo estático, sem PII). Páginas com dados continuam protegidas por requireUser. <i>Condição:</i> Combinado com F2, ou página nova no grupo (app) sem requireUser.
INFORMATIVA	Postgres ao vivo (grants) / padrão Supabase:—	F4 · GRANT amplo de anon/authenticated nas tabelas public anon e authenticated têm SELECT/INSERT/UPDATE/DELETE/TRUNCATE em perfis, logs, métricas e catálogo. RLS sem policy para anon bloqueia o acesso hoje. <i>Explorável:</i> Não explorável via PostgREST enquanto RLS estiver ligado e sem policy para anon. Vira crítico se RLS for desligado ou surgir policy permissiva. <i>Condição:</i> RLS desabilitado ou policy nova to public/anon.
INFORMATIVA	supabase/migrations/001_initial.sql:66–81	F5 · Função SECURITY DEFINER no schema public handle_new_user() é SECURITY DEFINER em public. EXECUTE está só em postgres e service_role (não em anon/authenticated). <i>Explorável:</i> Não chamável pelo cliente autenticado. Risco residual se grants forem ampliados. <i>Condição:</i> GRANT EXECUTE a anon/authenticated/public.

Categoria 2 — permissão no navegador

Não se aplica como falha: o produto não tem papéis. A UI de “Resetar dados” só esconde o segundo clique de confirmação; `resetUserData` no servidor exige sessão e apaga só `user.id`. Troca de senha no cliente chama Auth com a senha atual e depois `updateUser` — o privilégio é a sessão, não um flag `isAdmin`.

Categoria 3 — IDOR (cobertura)

Handlers percorridos: `GET src/app/auth/callback/route.ts`; actions `saveWorkoutLog`, `saveBodyMetrics`, `updateProfile`, `resetUserData`; páginas `/hoje`, `/historico`, `/perfil`, `/treino/[code]`, `/descanso`. Nenhuma busca/altera/apaga registro de outro usuário por ID. `getProfile/getWorkoutLogs/getBodyMetrics` aceitam `userId`, mas os callers passam `user.id` e o RLS corta o resto.

Categoria 5 — XSS (cobertura)

Não há lib de sanitização (`DOMPurify` etc.) e não foi necessário: não há sink de HTML com input do usuário. `ExerciseMedia` usa URLs do catálogo (escrita bloqueada por RLS). `AuthHeroBackground` e `BottomNav` usam paths estáticos. E-mails de reset são do Supabase Auth, fora deste repositório.

4. Recomendações priorizadas

- 1 **P1 — F1.** Ativar Leaked Password Protection no Supabase Auth.
- 2 **P2 — F2/F3.** Proxy fail-closed sem env; `requireUser` no layout (app); decidir se `/descanso` é público.
- 3 **P3 — F4.** REVOKE de privilegios de anon nas tabelas de usuário (manter RLS).
- 4 **P4 — F5.** Manter EXECUTE restrito; opcional mover `handle_new_user` para schema privado.
- 5 **P5.** Inicializar git (ou confirmar remote) para passar a auditar histórico de segredos.
- 6 **P6.** Validação de startup que recuse placeholders de `.env.example`.

5. Issues para o GitHub

Texto completo, pronto para colar. Achados triviais do mesmo tema foram agrupados (F2+F3+validação de env; F4+F5).

--- ISSUE 1 ---

[Segurança] Ativar proteção contra senhas vazadas no Supabase Auth

Labels sugeridas: `security`, `média`

Problema

O advisor de segurança do projeto Supabase (`beaokkxdhdpassarldzf`) reporta *Leaked Password Protection Disabled*. O app só

Por que é explorável

No signup (`SignupForm`) e na troca/redefinição de senha, uma senha já vazada (ex.: listas do HaveIBeenPwned) é aceita se t.

Evidência

- Advisor: `auth\_leaked\_password\_protection` (WARN, facing EXTERNAL)
- Remediação oficial: <https://supabase.com/docs/guides/auth/password-security#password-strength-and-leaked-password-protection>
- `src/lib/validation.ts` linhas 51-56 – apenas comprimento

Impacto

Contas com senha fraca/vazada; risco médio em app pessoal, maior se o e-mail for reutilizado.

Sugestão de correção

1. No dashboard Supabase → Authentication → Settings, ativar *Leaked password protection*.
2. Opcional: política de senha mais forte (mínimo 10-12, ou check no servidor).

Critérios de aceite

- [] Advisor `auth\_leaked\_password\_protection` deixa de aparecer (ou fica resolvido)
- [] Signup com senha conhecida como vazada é rejeitado pelo Auth

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Proxy de auth falha aberto e layout (app) não revalida sessão

Labels sugeridas: `security`, `baixa`

Problema

Três pontos fracos se somam: o proxy não autentica se as env do Supabase faltarem; o layout autenticado não chama `requireUser`

Por que é explorável

Com `NEXT\_PUBLIC\_SUPABASE\_URL` ou `NEXT\_PUBLIC\_SUPABASE\_ANON\_KEY` ausentes, `updateSession` devolve `NextResponse.next()` so

Evidência

...

// src/lib/supabase/proxy.ts:10-12

```
if (!url || !anonKey) {  
  return supabaseResponse;  
}
```

...

...

// src/app/(app)/layout.tsx – sem getUser/requireUser

```
export default function AppLayout({ children }) { ... }
```

...

...

// src/app/(app)/descanso/page.tsx:8

```
export default function DescansoPage() { // sem requireUser
```

...

`src/lib/supabase/client.ts:3-8` e `server.ts` usam `!` sem validar placeholder.

Impacto

Baixo no estado atual: `/hoje`, `/historico`, `/perfil`, `/treino/[code]` ainda redirecionam via `requireUser`. Produção ho

Sugestão de correção

1. Se env faltar: falhar fechado (503 ou redirect `/login`), não `next()`.
2. Validar URL/key no boot (rejeitar `YOUR\_PROJECT` / `your\_anon\_key`).
3. `requireUser()` no layout `(app)` (ou helper compartilhado).

Critérios de aceite

- [] Teste com env vazia: rotas `(app)` não entregam 200 autenticado
- [] `/descanso` exige sessão ou sai do grupo autenticado de propósito documentada
- [] Layout `(app)` chama `getUser()` / `requireUser()`

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Endurecer grants de anon e função SECURITY DEFINER

****Labels sugeridas:**** `security`, `informativa`

Problema

- 1) `anon` e `authenticated` têm GRANT de SELECT/INSERT/UPDATE/DELETE/TRUNCATE em todas as tabelas `public` (padrão Supabase)
- 2) `handle\_new\_user()` é SECURITY DEFINER no schema `public` (`001\_initial.sql:66-81`). EXECUTE atual: só `postgres` e `serv

Por que é explorável

Não é explorável hoje: RLS não tem policy para `anon`; a função não é executável pelo cliente. Fica perigoso se RLS for des

Evidência

- `information\_schema.role\_table\_grants` – anon ALL em profiles, workout_logs, body_metrics, exercises, workouts, workout_e
- `pg\_policies` – policies apenas `TO authenticated`
- `supabase/migrations/001\_initial.sql:66-81` – `security definer` + `set search\_path = public`

Impacto

Informativo (defesa em profundidade). Sem mudança de config, o Data API continua bloqueado para anônimo.

Sugestão de correção

``sql

revoke all on all tables in schema public from anon;

grant select on public.exercises, public.workouts, public.workout_exercises to authenticated;

-- manter DML só nas tabelas do usuário, se quiser restringir ainda mais

``

Mover `handle\_new\_user` para schema privado (ex.: `private`) ou manter EXECUTE restrito (já está).

Critérios de aceite

- [] `anon` sem GRANT de DML/TRUNCATE nas tabelas de dados do usuário
- [] RLS continua passando nos fluxos autenticados (hoje, perfil, log, métricas)
- [] EXECUTE de `handle\_new\_user` não está em `anon` nem `authenticated`

--- FIM ISSUE 3 ---